



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

CPU 架构相关编程

姓名：吕昊远

学号：2410668

专业：计算机科学与技术

指导教师：王刚

2026 年 4 月 3 日

目录

一、 实验背景	1
(一) 背景介绍	1
(二) 平台与环境	1
二、 任务 1: $N \times N$ 矩阵与向量内积	1
(一) 实验内容	1
(二) 程序设计	1
(三) 实验结果与分析	3
三、 任务 2: N 个数求和	5
(一) 实验内容	5
(二) 程序设计	5
(三) 实验结果与分析	6
四、 总结与体会	8
五、 附录: 代码仓库链接	8

一、实验背景

(一) 背景介绍

当今社会, 计算机技术飞速发展, 计算能力在超级计算、人工智能等前沿领域的重要性与日俱增。随着后摩尔时代的到来, 芯片工艺与制程精度的提升陷入瓶颈, 算力增长的重心开始转向计算机体系结构的优化与算法层面的改进。

本次实验将初步探究如何通过不同算法对程序进行优化设计, 结合 CPU 体系结构相关知识提升程序的运算效率; 同时借助 AMDuProf 性能分析工具, 采集 CPU 底层运行数据, 对比不同优化方式下的性能差异并分析其内在原因。

(二) 平台与环境

- 平台: Windows11 64 位 X86
- CPU: AMD Ryzen 7 7735H with Radeon Graphics (3.20 GHz)
- 开发环境: CodeBlocks
- 编译器: TDM-GCC
- 分析软件: AMDuProfPcm
- 系统计时器:

Windows 高精度计时

```
1 LARGE_INTEGER get_counter() {  
2     LARGE_INTEGER cnt;  
3     QueryPerformanceCounter(&cnt);  
4     return cnt;  
5 }
```

二、任务 1: N*N 矩阵与向量内积

(一) 实验内容

计算给定 N*N 矩阵的每一列与给定向量的内积, 考虑两种算法设计思路:

- a) 逐列访问元素的平凡算法;
- b) cache 优化算法;

在此基础上, 还可以对于 cache 优化算法进一步优化, 本实验中采用循环展开 (unroll) 实现。

(二) 程序设计

1. 平凡算法

按照矩阵与向量相乘的数学定义实现, 逐列遍历矩阵, 对每一列与向量进行点积运算, 使用最基础的循环嵌套实现计算逻辑, 时间复杂度为 $O(n^2)$ 。

逐列访问平凡算法

```
1  for (i = 0; i < n; i++) {
2      sum[i] = 0.0;
3      for (j = 0; j < n; j++) {
4          sum[i] += b[j][i] * a[j];
5      }
6  }
```

2. cache 优化算法

采用逐行访问矩阵元素的方式，每一步外层循环不计算出任何一个内积的最终结果，而只是向每个内积累加一个乘积。这样的访存模式具有很好空间局部性，使得 cache 的作用得到发挥，虽然时间复杂度仍为 $O(n^2)$ ，但计算效率得到提升。

逐行访问 cache 优化算法

```
1  for (i = 0; i < n; i++) {
2      sum[i] = 0.0;
3  }
4  for (j = 0; j < n; j++) {
5      for (i = 0; i < n; i++) {
6          sum[i] += b[j][i] * a[j];
7      }
8  }
```

3. cache 优化 + 循环展开

在 cache 优化算法逐行访存的基础上，进一步引入循环展开进行优化。通过四路循环展开，减少循环迭代次数、分支判断与循环控制指令的开销，充分利用 CPU 超标量提升指令级并行度；同时保留逐行访问的空间局部性，发挥 cache 的作用。该优化在不改变 $O(n^2)$ 时间复杂度的情况下，进一步降低了程序运行的额外开销，实现了运算效率的再次提升。

cache 优化 + 四路循环展开

```
1  for (i = 0; i < n; i++) {
2      sum[i] = 0.0;
3  }
4  for (j = 0; j < n; j++) {
5      // 4路循环展开，降低循环开销
6      for (i = 0; i + 3 < n; i += 4) {
7          sum[i] += b[j][i] * a[j];
8          sum[i+1] += b[j][i+1] * a[j];
9          sum[i+2] += b[j][i+2] * a[j];
10         sum[i+3] += b[j][i+3] * a[j];
11     }
12     // 处理剩余元素
13     for (; i < n; i++) {
14         sum[i] += b[j][i] * a[j];
15     }
16 }
```

```

15     }
16 }

```

(三) 实验结果与分析

测试代码

```

1  int idx;
2  for (idx = 0; idx < test_num; idx++) {
3      int n = sizes[idx];
4      int i, j;
5      // 动态分配内存
6      double** b = new double*[n];
7      for (i = 0; i < n; i++) {
8          b[i] = new double[n];
9      }
10     double* a = new double[n];
11     double* sum = new double[n];
12
13     init_data(n, b, a);
14
15     // 平凡算法
16     LARGE_INTEGER s1 = get_counter();
17     for (i = 0; i < 100; i++) plain(n, b, a, sum);
18     LARGE_INTEGER e1 = get_counter();
19     double t_plain = get_time(s1, e1, freq) / 100.0;
20     // Cache优化算法
21     LARGE_INTEGER s2 = get_counter();
22     for (i = 0; i < 100; i++) cache_opt(n, b, a, sum);
23     LARGE_INTEGER e2 = get_counter();
24     double t_cache = get_time(s2, e2, freq) / 100.0;
25     // 循环展开算法
26     LARGE_INTEGER s3 = get_counter();
27     for (i = 0; i < 100; i++) cache_unroll(n, b, a, sum);
28     LARGE_INTEGER e3 = get_counter();
29     double t_unroll = get_time(s3, e3, freq) / 100.0;
30
31     // 输出结果
32     out << "N=" << n << endl;
33     out << "平凡算法:" << t_plain << "us" << endl;
34     out << "Cache优化:" << t_cache << "us" << endl;
35     out << "循环展开:" << t_unroll << "us" << endl;
36     out << "-----" << endl;
37     // 释放内存
38     for (i = 0; i < n; i++) delete[] b[i];
39     delete[] b;
40     delete[] a;
41     delete[] sum;

```

42

}

使用 C++ 调用 Windows 高精度计时的 API 统计运行时间 (QueryPerformanceCounter), 为了减小误差, 在主函数中分别对上述三种算法各自执行 100 次得到平均时间。程序运行完毕后, 将运行时间结果输出为文件 “mmv_result.txt” 保存, 使用 Python 编写可视化代码分析结果文件, 得到如下图像。

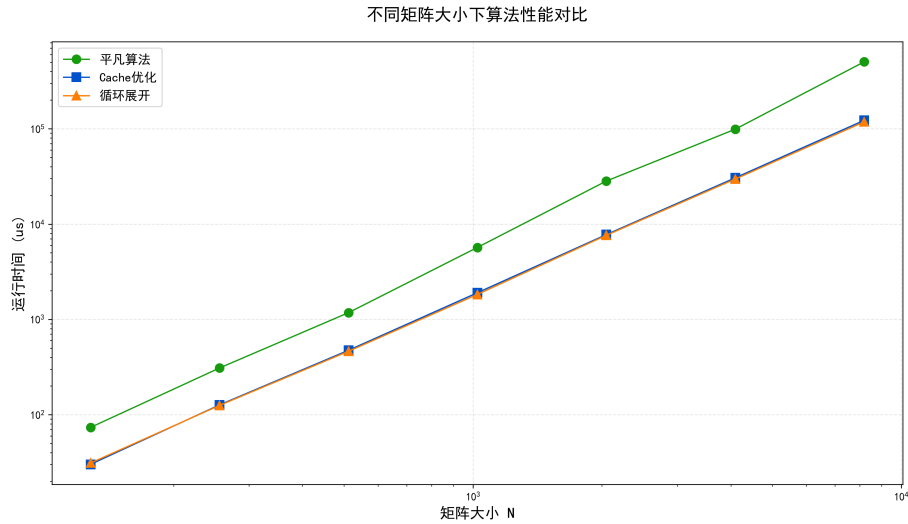


图 1: 三种算法性能对比

如图1可知, cache 优化效果显著, 运行时间大幅减少; 而循环展开的进一步优化则收效甚微, 虽然对比单独的 cache 优化算法确实有所提升, 但其效果并不明显。为了进一步探讨背后的原因, 使用 AMDuProf 进行硬件层面的数据采集, 得到如下数据。

数据项	Plain 平凡算法	Cache 优化算法	Cache+Unroll
IPC (Sys+User)	1.34	2.17	2.68
L2 Access (pti)	112.78	26.38	17.77
L2 Miss (pti)	71.07	9.93	6.68
IC Miss (pti)	8.20	10.22	4.43
L3 Access (pti)	59.75	9.57	6.72
L3 Miss (pti)	2.92	3.92	4.48
L3 Miss (%)	4.88%	44.68%	67.15%
CPU 频率 (MHz)	3370	3710	3865

表 1: AMDuProf 测试结果

从 AMDuProf 测试得到的硬件数据可以看出, 三种算法的性能差异主要体现在底层硬件行为上。平凡算法 IPC 仅 1.34, 根源在于逐列访存引发严重的流水线阻塞, 同时 L2 缓存缺失严重 (71.07 pti), CPU 长期等待内存, 大量运算资源空闲; Cache 优化后 IPC 大幅提升至 2.17, L2 访问与缺失数骤降, 在空间局部性上的优化极大解决了访存的问题, 充分释放了 CPU 算力; 循环展开进一步将 IPC 推至 2.68, 通过减少循环开销来提升指令级的并行程度, 在高效访存的基础上又优化了执行效率。同时, 还注意到在 L3 部分出现缺失率异常, 通过分析得知, L3 Miss

总量并未发生太大波动，而总访问量大幅下降，导致缺失率升至 44.68%、67.15%，但是这实际上是有效访问占比提升带来的变化，反而印证了访存模式的优化效果。

CPU 频率随优化程度递增 (3370→3710→3865MHz)，是动态睿频对高负载的正常响应。优化后的算法指令执行更紧凑、内存等待时间更短，单位时间内指令吞吐量更大，负载强度更高，因此触发了更高的睿频状态，进一步放大了优化带来的实际收益。Cache 优化解决了程序的核心访存瓶颈，因此能带来量级式的性能提升；循环展开则是在访存瓶颈已基本消除的基础上，进一步优化指令级并行，其优化空间本就相对有限。

三、任务 2: N 个数求和

(一) 实验内容

计算 N 个数的和，考虑两种算法设计思路：

a) 逐个累加的平凡算法（链式）；

b) 超标量优化算法（指令级并行），如最简单的两路链式累加；再如递归算法——两两相加、中间结果再两两相加，依次类推，直至只剩下最终结果；

在此基础上，本实验中还进一步优化了并行算法，继续尝试四路并行优化，以便对比不同并行度下的性能差异，验证并行度对运算效率的提升效果。

(二) 程序设计

1. 平凡算法

使用 for 循环将 N 个变量的值遍历累加，时间复杂度为 $O(n)$ 。

单链路平凡算法

```
1  int i;  
2  sum = 0.0;  
3  for (i = 0; i < n; i++) {  
4      sum += a[i];  
5  }
```

2. 两路优化算法

采用两路指令级并行，通过定义两个独立的累加变量 sum1、sum2 优化单链式累加的数据依赖问题。循环以步长 2 遍历数组，同时对两个数组元素分别累加，让 CPU 可以并行执行两条加法指令。

两路并行优化算法

```
1  int i;  
2  double sum1 = 0.0, sum2 = 0.0;  
3  for (i = 0; i < n; i += 2) {  
4      sum1 += a[i];  
5      sum2 += a[i + 1];  
6  }  
7  sum = sum1 + sum2;
```

3. 四路优化算法

将两路并行提升至四路并行，进一步提升指令级并行度。定义四个相互独立的累加变量 sum1 sum4，循环以步长 4 遍历数组，实现四条加法指令的并行执行，更大程度地利用 CPU 超标量的并行计算能力。循环结束后单独处理剩余元素，保证数据累加的完整性，最终合并四个中间结果得到总和，并行效率较两路算法进一步提升。

四路并行优化算法

```

1  int i;
2  double sum1 = 0.0, sum2 = 0.0;
3  double sum3 = 0.0, sum4 = 0.0;
4  for (i = 0; i + 3 < n; i += 4) {
5      sum1 += a[i];
6      sum2 += a[i+1];
7      sum3 += a[i+2];
8      sum4 += a[i+3];
9  }
10 // 处理剩余元素
11 for (; i < n; i++) {
12     sum1 += a[i];
13 }
14 sum = sum1 + sum2 + sum3 + sum4;
```

(三) 实验结果与分析

测试代码

```

1  int idx;
2  for (idx = 0; idx < test_num; idx++) {
3      int n = sizes[idx];
4      int i;
5      double* a = new double[n];
6      double res;
7      init_arr(n, a);
8
9      // 平凡算法
10     LARGE_INTEGER s1 = get_counter();
11     for (i = 0; i < 1000; i++) sum_plain(n, a, res);
12     LARGE_INTEGER e1 = get_counter();
13     double t_plain = get_time(s1, e1, freq) / 1000.0;
14     // 两路优化算法
15     LARGE_INTEGER s2 = get_counter();
16     for (i = 0; i < 1000; i++) sum_opt(n, a, res);
17     LARGE_INTEGER e2 = get_counter();
18     double t_opt = get_time(s2, e2, freq) / 1000.0;
19     // 4路循环展开
20     LARGE_INTEGER s3 = get_counter();
21     for (i = 0; i < 1000; i++) sum_unroll(n, a, res);
```



```

22     LARGE_INTEGER e3 = get_counter();
23     double t_unroll = get_time(s3, e3, freq) / 1000.0;
24
25     // 输出结果
26     out << "N_=" << n << endl;
27     out << "平凡算法:" << t_plain << "us" << endl;
28     out << "两路优化:" << t_opt << "us" << endl;
29     out << "4路循环展开:" << t_unroll << "us" << endl;
30     out << "-----" << endl;
31     delete[] a;
32 }

```

使用 C++ 调用 Windows 高精度计时的 API 统计运行时间 (QueryPerformanceCounter), 为了减小误差, 在主函数中分别对上述三种算法各自执行 1000 次得到平均时间。程序运行完毕后, 将运行时间结果输出为文件 “sum_result.txt” 保存, 使用 Python 编写可视化代码分析结果文件, 得到如下图像。

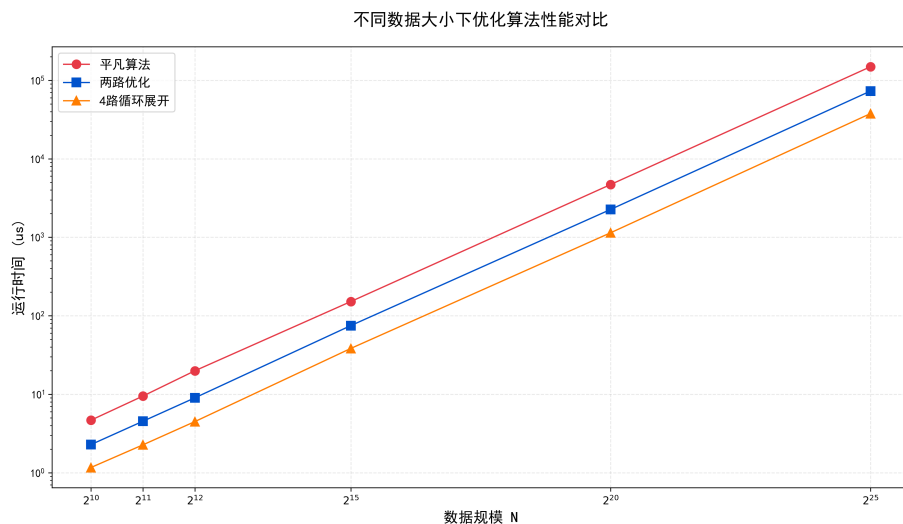


图 2: 三种算法性能对比

如图2可知, 平凡算法的运行耗时最长, 运算效率最低; 两路并行优化算法耗时显著降低, 性能较平凡算法有明显提升; 四路并行优化算法耗时最短, 运算效率达到最优。随着指令级并行度的提升, 超标量优化显著提升了运算效率。从数据来看, 三种算法在对数坐标上截距等差, 符合算法中成倍提高并行度的优化逻辑。为了进一步分析, 使用 AMDuProf 进行硬件层面的数据采集, 得到如下数据。

从 AMDuProf 测试得到的硬件数据可以看出, 三种算法主要在底层的并行度上具有差异。平凡算法的 IPC 仅 0.63, 根源在于单链式累加引发严重的数据依赖与流水线阻塞, CPU 计算资源大量闲置无法充分利用; 两路并行优化后 IPC 提升至 0.91, 通过双累加器优化了数据依赖与流水线停顿问题, 指令执行效率明显改善; 四路并行进一步将 IPC 推高至 1.54, 依靠多累加器设计充分发挥 CPU 超标量的指令级并行能力, 大幅提升了指令吞吐量。

指令缓存缺失 (IC Miss) 从 16.82 逐步降至 7.89, 循环展开增大了循环体有效指令长度、减少了循环分支跳转开销, 提升了指令缓存命中率, 进一步降低了取指等待带来的性能损耗。同时, 与任务一中的矩阵计算类似, L3 缓存缺失率出现了上升趋势, 而两者原因本质相同, 均为总访

数据项	Plain 平凡算法	两路优化算法	四路优化算法
IPC (Sys+User)	0.63	0.91	1.54
L2 Access (pti)	38.20	36.94	32.01
L2 Miss (pti)	16.34	16.44	14.48
IC Miss (pti)	16.82	11.66	7.89
L3 Access (pti)	16.81	16.96	15.61
L3 Miss (pti)	8.85	11.18	11.98
L3 Miss (%)	53.32%	66.06%	76.89%
CPU 频率 (MHz)	4130	4040	3962

表 2: AMDuProf 测试结果

问量减小导致的。

CPU 频率随并行度提升略有降低，经过多方面分析，推测为高并行负载下功耗与温度管控引发的正常降频，波动幅度较小，其负面影响远不及 IPC 与指令并行度提升带来的性能提升。单链式累加的数据依赖是本次求和运算的核心性能瓶颈，两路并行初步解除阻塞，四路并行则充分释放了 CPU 超标量算力，最终实现与并行度匹配的稳定性提升，与运行耗时的数据图像完全符合。

四、 总结与体会

本次实验的核心目的是掌握 CPU 并行程序优化的基本思路，理解 CPU 体系结构中 cache 缓存、超标量运算、指令级并行等对程序性能的影响，学会运用性能分析工具采集数据，并对优化效果进行分析验证。实验分为两个独立任务，各设计了三种不同优化程度的算法，使用精准计时、数据可视化和硬件性能数据采集等方式，较为全面的分析得出实验结论。

在任务 1（矩阵与向量内积）中，主要通过 cache 优化提升了计算的空间局部性，大幅降低了缓存缺失率，释放 CPU 算力，带来了量级式的性能提升；核心的访存问题已被 cache 优化解决后，剩余优化空间较小。体现出 cache 在 CPU 运行时的对于访存的重要程度。

在任务 2（N 个数求和）中，采用并行优化来提升指令级并行度，显著降低了运行耗时。通过对于数据依赖与流水线的优化，充分利用 CPU 超标量特性，提升整体执行效率，达到性能的成倍增强。

综合两个任务的实验结果可以发现，程序性能优化的核心在于找准性能瓶颈：任务一的瓶颈是访存效率，任务二的瓶颈是数据依赖与指令并行度，针对性地结合 CPU 体系结构特点设计优化算法，才能实现性能的有效提升；同时，性能优化也并非盲目叠加优化手段，需结合瓶颈所在，合理地选择优化方式，才能达到最佳效果。在分析程序时，AMDuProf 等性能测试工具的运用，也为硬件视角下的分析提供了数据依据，便于理解不同算法下优化效果的硬件逻辑本质。

五、 附录：代码仓库链接

GitHub: <https://github.com/Iriasoul/Parallel-Programming/tree/main/lab01>